

```
In[4]: sp.integrate.quad(lambda x: 2**sqrt(x)/sqrt(x),1,3)
```

```
Out[4]: (3.8144772785946079, 4.2349205016052412e-14)
```

The first argument to `quad` is a ‘callable’ Python object (i.e., a function, method, or class instance). We have used a lambda function as the argument in this case. (A lambda function is one that takes any number of arguments—including optional arguments—and returns the value of a single expression.) The next two arguments are the limits of integration. The return value is a tuple, with the first element holding the estimated value of the integral, and the second element holding an upper bound on the error.

Differentiation

We can get a derivative at a point via automatic differentiation, supported by `FuncDesigner` and `OpenOpt`, which are scientific packages based on `SciPy`. Note that automatic differentiation is different from symbolic and numerical differentiation. In symbolic differentiation, the function is differentiated as an expression, and is then evaluated at a point. Numerical differentiation makes use of the method of finite differences. However, automatic differentiation is the decomposition of differentials provided by the chain rule. A complete understanding of automatic differentiation is beyond the scope of this article, so I’d recommend that interested readers refer to Wikipedia. Automatic differentiation works by decomposing the vector function into elementary sequences, which are then differentiated by a simple table lookup.

Unfortunately, a deeper understanding of automatic differentiation is required to make full use of the scientific packages provided in Python. Hence, in this article, we’ll focus on symbolic differentiation, which is easier to understand and implement. We’ll be using a powerful computer algebra system known as *Maxima* for symbolic differentiation. *Maxima* is a version of the MIT-developed *MACSYMA* system, modified to run under *CLISP*. Written in *Lisp*, it allows differentiation, integration, solutions for linear or polynomial equations, factoring of polynomials, expansion of functions in the Laurent or Taylor series, computation of the Poisson series, matrix and tensor manipulations, and two- and three-dimensional graphics.

Open the Synaptic Package Manager and install the *maxima* package. Once installed, you can run it by executing the *maxima* command in the terminal. We’ll be differentiating the following simple functions with the help of *Maxima* (Figure 2 displays *Maxima* in action):

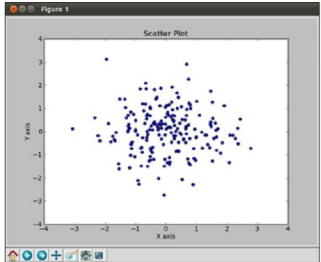
$$\begin{aligned} & d/dx (x^4) \\ & d/dx (\sin x + \tan x) \\ & d/dx (1/\log x) \end{aligned}$$


Figure 9: Scatter Plot with Matplotlib

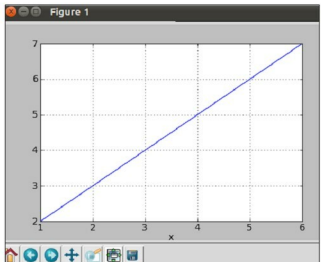


Figure 10: Plotting by fetching data from the text file

You have to simply define the function in `diff()` and *maxima* will calculate the derivative for you.

```
1  (%i1) diff(x^4)
2
3  (%o1) 4x^3 del(x)
4
5  (%i2) diff(sin(x) + tan(x))
6
7  (%o2) (sec^2(x) + cos(x))del(x)
8
9  (%i3) diff(1/log(x))
10
11 (%o3) - del(x)/x log^2(x)-
```

The command `diff(expr,var,num)` will differentiate the expression in Slot 1 with respect to the variable entered in